

ESCOLA DE NEGÓCIOS, TECNOLOGIA E INOVAÇÃO DO CESUPA

Inteligência Artificial e Computacional (0700M8)

Prof. Daniel Leal Souza — Semestre 01/2026

# Sims Routine Optimizer

*Otimização de Rotina Diária com Algoritmo Genético e PSO*

## RELATÓRIO TÉCNICO — OPÇÃO 2: PROTÓTIPO DE PROGRAMA

### Equipe

Andrey Lourival Andrade Garcia

Everton de Oliveira da Silva

Filipe César Maciel Sucupira

Igor Cecim Vilhena

Belém, Pará — Junho de 2026

## 1. Identificação do Projeto

---

| Campo        | Informação                                                                                                                  |
|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| Projeto      | Sims Routine Optimizer                                                                                                      |
| Disciplina   | Inteligência Artificial e Computacional (0700M8)                                                                            |
| Professor    | Prof. Daniel Leal Souza                                                                                                     |
| Instituição  | CESUPA — Escola de Negócios, Tecnologia e Inovação                                                                          |
| Semestre     | 01/2026                                                                                                                     |
| Modalidade   | Opção 2 — Protótipo de Programa                                                                                             |
| GitHub       | <a href="https://github.com/andreygarcia/sims-routine-optimizer">https://github.com/andreygarcia/sims-routine-optimizer</a> |
| Integrante 1 | Andrey Lourival Andrade Garcia                                                                                              |
| Integrante 2 | Everton de Oliveira da Silva                                                                                                |
| Integrante 3 | Filipe César Maciel Sucupira                                                                                                |
| Integrante 4 | Igor Cecim Vilhena                                                                                                          |

## 2. Problema e Público-Alvo

### 2.1 Descrição do Problema

O Sims Routine Optimizer resolve um problema de otimização combinatorial inspirado no jogo The Sims: dado um personagem (Sim) com traços de personalidade definidos, qual sequência de atividades ao longo de 24 horas maximiza a sua felicidade geral, respeitando o decaimento natural de suas oito necessidades biológicas e psicológicas?

O Sim possui oito necessidades que decaem passivamente ao longo do tempo — fome, energia, diversão, social, higiene, banheiro, conforto e ambiente — cada uma com peso distinto na felicidade final. O desafio é distribuir 48 blocos de 30 minutos entre 20 atividades disponíveis de maneira ótima para o perfil específico do personagem.

Do ponto de vista formal, trata-se de um problema de sequenciamento com restrições de dependência temporal, decaimento dinâmico de variáveis de estado e função objetivo não-linear. Com 20 atividades possíveis por bloco e 48 blocos diários, o espaço de busca possui mais de  $20^{48}$  combinações possíveis — tornando a busca exaustiva completamente inviável.

| Necessidade | Peso na Felicidade |
|-------------|--------------------|
| Fome        | 20%                |
| Energia     | 20%                |
| Diversão    | 15%                |
| Social      | 15%                |
| Higiene     | 10%                |
| Banheiro    | 10%                |
| Conforto    | 5%                 |
| Ambiente    | 5%                 |

### 2.2 Público-Alvo

O sistema destina-se primariamente a estudantes e entusiastas de jogos de simulação de vida que queiram compreender, de forma interativa, como algoritmos evolutivos podem otimizar decisões de planejamento cotidiano. O contexto lúdico do The Sims serve de veículo para conceitos de computação evolutiva.

A formulação do problema é análoga a sistemas reais de escalonamento de atividades com múltiplos objetivos conflitantes, decaimento de recursos e restrições contextuais — como planejamento de rotinas de saúde, otimização de agendas de produtividade ou escalonamento de tarefas em sistemas embarcados.

### 2.3 Decisão Apoiada pelo Sistema

O sistema apoia a decisão de qual rotina diária adotar para maximizar o bem-estar do Sim. O usuário informa os traços do personagem, configura os parâmetros do algoritmo, executa a otimização e recebe uma rotina completa de 24 horas com visualização do estado das necessidades, curva de convergência e comparação entre AG e PSO.

### 2.4 Por que Computação Evolutiva?

- Espaço de busca combinatorial e exponencial:  $20^{48}$  combinações tornam a busca exaustiva inviável. Qualquer método exato escalaria de forma proibitiva para instâncias de 48 blocos.
- Função objetivo não-linear e dependente de sequência: o valor de uma atividade depende do estado atual das necessidades e dos traços do Sim, tornando métodos analíticos inadequados.

- Restrições de consistência estrutural: atividades multi-bloco (ex: dormir = 16 blocos consecutivos) exigem mecanismos de reparo que os operadores evolutivos acomodam naturalmente.
- Interações entre traços e categorias: o espaço de fitness é multimodal e irregular, beneficiando-se da diversidade populacional do AG e da memória coletiva do PSO.

## 3. Requisitos do Sistema

### 3.1 Requisitos Funcionais

1. Permitir seleção de perfil de Sim entre quatro opções pré-definidas (Workaholic, Social, Criativo, Equilibrado) ou configuração manual de traços.
2. Permitir configuração dos parâmetros do AG: população, gerações, taxa de crossover, taxa de mutação, tamanho do torneio e elitismo.
3. Permitir configuração dos parâmetros do PSO: tamanho do enxame, iterações, inércia  $w$ , coeficientes cognitivo  $c1$  e social  $c2$ .
4. Executar o AG e/ou PSO e retornar a melhor rotina encontrada (lista de 48 atividades com horários).
5. Exibir a curva de convergência do fitness ao longo das gerações/iterações.
6. Exibir o estado final das 8 necessidades do Sim em forma de gráfico de radar ou barras.
7. Permitir comparar AG e PSO lado a lado com métricas de desempenho.
8. Exportar a rotina otimizada em formato CSV e como relatório HTML completo. (Alternativa 1 — Pontuação Extra)
9. Executar a suite de testes automatizados com 5 execuções  $\times$  4 perfis e salvar resultados em JSON.

### 3.2 Requisitos Não Funcionais

- Desempenho: cada execução deve concluir em menos de 10 segundos em hardware convencional (medido: 7,4–8,9 s por execução).
- Reprodutibilidade: execuções com a mesma semente devem produzir resultados idênticos (determinismo via `random.Random` com seed).
- Usabilidade: interface web via Streamlit, acessível no navegador sem configuração adicional após instalação.
- Portabilidade: compatível com Python 3.10+ em Windows, Linux e macOS.
- Manutenibilidade: código modular com separação clara entre modelo (`sim.py`, `activities.py`), motores evolutivos (`genetic_algorithm.py`, `pso.py`), avaliação (`fitness.py`) e interface (`app.py`).

### 3.3 Entradas, Saídas e Fluxo de Uso

| Tipo    | Item                    | Descrição                                                       |
|---------|-------------------------|-----------------------------------------------------------------|
| Entrada | Perfil do Sim           | Traços de personalidade e pesos (dicionário Python)             |
| Entrada | Parâmetros AG/PSO       | Tamanho de população/enxame, gerações/iterações, taxas, semente |
| Entrada | Necessidades iniciais   | Estado inicial de cada necessidade (padrão: 0.0)                |
| Saída   | Rotina otimizada        | Lista de 48 atividades (cromossomo/posição do enxame)           |
| Saída   | Fitness e felicidade    | Valor da função objetivo e felicidade média normalizada 0–100%  |
| Saída   | Curva de convergência   | Histórico de melhor e média de fitness por geração/iteração     |
| Saída   | Estado das necessidades | Valores finais e médios das 8 necessidades ao longo do dia      |
| Saída   | CSV / HTML exportável   | Relatório completo da rotina otimizada para download            |

#### Fluxo de Uso

10. Executar: `streamlit run app.py` e acessar `http://localhost:8501` no navegador.
11. Selecionar o perfil do Sim e configurar os parâmetros do algoritmo na barra lateral.
12. Escolher o algoritmo desejado: AG, PSO ou ambos para comparação lado a lado.

13. Clicar em executar; barra de progresso exhibe o avanço por geração/iteração em tempo real.
14. Visualizar a rotina gerada com horários e emojis das atividades, gráfico de necessidades e curva de convergência.
15. Exportar em CSV ou relatório HTML se desejado.

### 3.4 Limitações e Riscos de Interpretação

- O modelo de necessidades é simplificado; as taxas de decaimento são fixas, não replicando exatamente a mecânica do The Sims original.
- O estado inicial das necessidades nos testes automatizados é fixo em 0, o que não reflete cenários onde o personagem inicia o dia com necessidades parcialmente satisfeitas.
- Atividades multi-bloco são tratadas como atômicas; não é possível interromper uma atividade no meio.
- O AG e o PSO podem convergir para ótimos locais em execuções com sementes desfavoráveis, especialmente em perfis com espaço de busca irregular.

## 4. Formulação da Otimização

### 4.1 Variáveis de Decisão e Representação

Cada solução candidata é representada como um cromossomo (AG) ou posição (PSO):

$$R = [a_1, a_2, \dots, a_{48}]$$

onde cada  $a_i \in A$  é o nome de uma atividade do catálogo A, e i representa o i-ésimo bloco de 30 minutos do dia (0h–24h). Atividades com duração D blocos ocupam D posições consecutivas com o mesmo valor — restrição estrutural mantida pelo operador `repair()`.

### 4.2 Restrições

#### Hard Constraints — penalidades severas na função de aptidão

- Consistência de blocos: atividades multi-bloco devem ser contíguas. Garantida pelo `repair()` após qualquer operador genético.
- Sono obrigatório: ausência de dormir ou soneca penaliza -100 pontos.
- Alimentação obrigatória: ausência de `comer_refeicao`, `lanche_rapido` ou `sair_jantar` penaliza -75 pontos.
- Banheiro obrigatório: ausência penaliza -40 pontos.
- Higiene mínima: ausência de banho penaliza -20 pontos.

#### Soft Constraints — penalidades graduais

- Necessidades críticas no final do dia: se `necessidade < -50`, penalidade =  $(-50 - \text{valor}) \times \text{peso} \times 10$ .
- Necessidades urgentes durante execução: se `hunger`, `bladder` ou `energy < -30` por bloco, penalidade =  $\text{déficit} \times \text{peso} \times 8$ .
- Penalidade contextual: atividade fora de condição ótima (ex: dormir com `energy > 20`, comer com `hunger > 40`) gera penalidade via `context_penalty()` proporcional ao desvio.
- Multiplicador de bem-estar: se necessidades básicas (`hunger`, `energy`, `hygiene`, `bladder`) estão negativas, a felicidade obtida de qualquer atividade é reduzida proporcionalmente. Fator mínimo: 0,1.

### 4.3 Função Objetivo / Aptidão

A função de aptidão  $f(R)$  implementada em `fitness.py`:

$$f(R) = \max(0, \sum \text{felicidade}(a_i, \text{traços}) \times \text{bem\_estar} + \text{bônus\_diversidade} + \text{bônus\_equilíbrio} + \text{bônus\_moodlets} - \text{penalidades\_totais} + \text{felicidade\_média\_diária} \times 10)$$

- $\sum \text{felicidade}(a_i, \text{traços}) \times \text{bem\_estar}$ : soma acumulada da felicidade de cada atividade ao longo dos 48 blocos, modulada pelos traços e pelo multiplicador de bem-estar. Cada atividade tem um vetor `needs_delta` com impacto em cada necessidade.
- `bônus_diversidade`: número de categorias distintas utilizadas  $\times 8$  pontos. Incentiva variedade entre trabalho, lazer, social, básico e cuidado.
- `bônus_equilíbrio`: para cada necessidade com valor  $\geq 0$  ao final do dia, soma `peso_necessidade`  $\times 20$ . Incentiva o Sim a terminar o dia equilibrado.
- `bônus_moodlets`: combinações de tags ativam moodlets — por exemplo, `{fitness, hygiene}` ativa "Corpo em Forma" (+15 pts). Há 8 moodlets implementados com bônus entre 9 e 16 pontos.
- `felicidade_média_diária`  $\times 10$ : média dos scores de felicidade instantânea ao longo dos 48 blocos, em escala 0–100, multiplicada por 10 para equivaler a uma contribuição de 0–1000 pontos. Este componente alinha o fitness com o bem-estar geral do Sim ao longo de todo o dia.

O sentido da otimização é maximização:  $f(R)$  maior indica melhor rotina.

### 4.4 Traços de Personalidade

Os traços modulam o impacto das atividades por categoria. Cada traço amplifica ou reduz a felicidade gerada por atividades de certas categorias:

| Traço               | Efeito nas categorias                    |
|---------------------|------------------------------------------|
| <b>workaholic</b>   | trabalho +50%, lazer -10%                |
| <b>introvertido</b> | social -50%, lazer +30%, básico +10%     |
| <b>extrovertido</b> | social +50%, lazer +10%                  |
| <b>criativo</b>     | lazer +40%, trabalho +10%                |
| <b>preguiçoso</b>   | básico +30%, trabalho -50%, cuidado -20% |
| <b>ativo</b>        | cuidado +50%, lazer +10%, básico -10%    |
| <b>gastrônomo</b>   | básico +40%                              |
| <b>sociável</b>     | social +40%, lazer +10%                  |
| <b>solitário</b>    | social -60%, básico +20%, lazer +20%     |

## 4.5 Métricas de Avaliação

| Métrica                         | O que mede                      | Como é calculada                                                                                        |
|---------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Fitness</b>                  | Qualidade global da solução     | $f(R)$ conforme formulação da seção 4.3                                                                 |
| <b>Felicidade Média (%)</b>     | Bem-estar médio ao longo do dia | Média de <code>happiness_score(needs)</code> para cada um dos 48 blocos                                 |
| <b>Melhoria vs Baseline (%)</b> | Ganho sobre rotina aleatória    | $\frac{(\text{média\_alg} - \text{média\_base})}{\max(\text{média\_base}, 1)} \times 100$               |
| <b>Desvio-Padrão</b>            | Estabilidade entre execuções    | DP amostral das 5 execuções por perfil                                                                  |
| <b>Curva de Convergência</b>    | Velocidade de melhoria          | Melhor e média do fitness por geração/iteração ( <code>history["best"]</code> e <code>["mean"]</code> ) |
| <b>Tempo de Execução (s)</b>    | Custo computacional             | Tempo wall-clock médio das 5 execuções ( <code>time.time()</code> )                                     |



## 5. Motor Evolutivo

### 5.1 Algoritmo Genético (genetic\_algorithm.py)

#### Representação cromossômica

Cada cromossomo é uma lista Python de 48 strings (nomes de atividades). A função `random_chromosome(rng)` gera indivíduos válidos percorrendo os 48 blocos sequencialmente e escolhendo atividades aleatórias do catálogo respeitando sua duração. A função `repair(chrom, rng)` percorre o cromossomo e reconstrói as sequências de atividades multi-bloco após qualquer operador que possa quebrar a consistência.

#### Seleção por Torneio (k=3)

A função `tournament_selection()` sorteia aleatoriamente k=3 índices da população e retorna o cromossomo com maior fitness entre eles. O torneio foi preferido à roleta por ser mais robusto a populações heterogêneas, não sofrer de pressão seletiva excessiva em convergência avançada e ter custo computacional  $O(k)$  versus  $O(n)$  da roleta.

#### Crossover de Dois Pontos

A função `two_point_crossover()` sorteia dois pontos p e q no intervalo [1, 47] e gera: `filho1 = pai1[:p] + pai2[p:q] + pai1[q:]` e `filho2 = pai2[:p] + pai1[p:q] + pai2[q:]`. O crossover de dois pontos foi escolhido por preservar melhor segmentos internos úteis em cromossomos longos de 48 genes, em comparação ao crossover de um ponto. Após o crossover, `repair()` é aplicado a ambos os filhos. Taxa: 85%.

#### Mutação

Aplicada por cromossomo com taxa de 12%. Dois tipos escolhidos aleatoriamente com igual probabilidade:

- `replace`: posição aleatória i é escolhida; nova atividade substitui a partir de i conforme a duração da atividade.
- `swap`: dois genes i e j são permutados. `repair()` é aplicado em ambos os casos.

#### Elitismo e Critério de Parada

Os 2 melhores indivíduos de cada geração são preservados intactos (`elitismo=2`). Critério de parada: número fixo de 150 gerações, escolhido pela previsibilidade de tempo de execução e por observação empírica de convergência antes desse limite.

| Parâmetro            | Valor | Justificativa                                                                                         |
|----------------------|-------|-------------------------------------------------------------------------------------------------------|
| Tamanho da população | 80    | Diversidade suficiente sem custo excessivo. Valores menores causavam convergência prematura.          |
| Gerações             | 150   | Convergência observada empiricamente antes de 150 gerações na maioria dos perfis.                     |
| Taxa de crossover    | 85%   | Alta recombinação promove exploração. Acima de 90% reduzia diversidade na convergência.               |
| Taxa de mutação      | 12%   | Por cromossomo. Mantém diversidade sem destruir boas soluções. Acima de 20% prejudicava convergência. |
| Torneio k            | 3     | Pressão seletiva moderada. k=2 insuficiente; k=5 causava convergência excessivamente rápida.          |
| Elitismo             | 2     | Preserva os 2 melhores por geração, evitando regressão sem prejudicar diversidade.                    |

### 5.2 PSO Discreto (pso.py)

#### Adaptação para Espaço Combinatorial

O PSO clássico (Kennedy & Eberhart, 1995) opera em espaços contínuos. Para o problema de sequenciamento discreto, foi implementada uma adaptação em que:

- Cada partícula representa uma rotina completa (lista de 48 genes — mesma representação do AG).
- A velocidade  $vel[i]$  de cada gene  $i$  é um valor contínuo em  $[-6, 6]$ , interpretado como tendência de mudança via função sigmóide:  $P(\text{mudança}) = \sigma(vel[i]) = 1 / (1 + e^{(-vel[i])})$ .
- A atualização de posição substitui o gene  $i$  com probabilidade  $\sigma(vel[i])$ : 45% de chance de adotar  $pbest[i]$ , 45% de adotar  $gbest[i]$ , e 10% de exploração aleatória.
- Após cada atualização de posição, `repair()` é aplicado para manter a consistência das atividades multi-bloco.

## Atualização de Velocidade

*$vel[i]_{\text{novo}} = w \times vel[i] + c1 \times r1 \times (pbest[i] \neq pos[i]) + c2 \times r2 \times (gbest[i] \neq pos[i])$  Clamped em  $[-6, 6]$  para evitar overflow no sigmoid.*

| Parâmetro            | Valor | Justificativa                                                                             |
|----------------------|-------|-------------------------------------------------------------------------------------------|
| Tamanho do enxame    | 80    | Equivalente ao AG para comparação justa em número total de avaliações da função objetivo. |
| Iterações            | 150   | Equivalente às gerações do AG: $80 \times 150 = 12.000$ avaliações por execução.          |
| Inércia $w$          | 0,7   | Valor clássico da literatura (Shi & Eberhart, 1998). Promove exploração no início.        |
| Coef. cognitivo $c1$ | 1,5   | Atração para o melhor histórico pessoal. Valor padrão da literatura para PSO discreto.    |
| Coef. social $c2$    | 1,5   | Atração para o melhor global. Balanceado com $c1$ para equilíbrio exploração/exploação.   |

## 5.3 Baseline: Rotina Aleatória

A função `random_baseline()` em `genetic_algorithm.py` gera uma rotina completamente aleatória usando a mesma semente informada, sem nenhuma lógica de planejamento. Serve como referência mínima — se os algoritmos evolutivos não superarem a busca aleatória, não têm utilidade prática.

## 6. Arquitetura do Protótipo

### 6.1 Estrutura Modular

| Módulo                                | Responsabilidade                                                                                                                                                                                                                                                         |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/activities.py</code>        | Catálogo das 20 atividades, constantes <code>NEED_KEYS</code> , <code>NEED_WEIGHTS</code> , <code>NEED_DECAY</code> , sistema de moodlets ( <code>MOODLETS</code> ) e classe <code>Activity</code> com <code>total_happiness()</code> e <code>context_penalty()</code> . |
| <code>src/sim.py</code>               | Modelo do Sim: classe <code>Sim</code> com traços, necessidades iniciais e 4 perfis pré-definidos via <code>Sim.from_profile()</code> . Inclui <code>happiness_score()</code> .                                                                                          |
| <code>src/fitness.py</code>           | Função <code>evaluate()</code> : simula o dia completo, aplica decaimento passivo, penalidades contextuais e estruturais, bônus de diversidade, equilíbrio e moodlets. Retorna ( <code>fitness</code> , <code>info_dict</code> ).                                        |
| <code>src/genetic_algorithm.py</code> | Motor do AG: <code>random_chromosome</code> , <code>repair</code> , <code>tournament_selection</code> , <code>two_point_crossover</code> , <code>mutate</code> , <code>run_ga</code> e <code>random_baseline</code> . Inclui <code>GACONFIG</code> dataclass.            |
| <code>src/psa.py</code>               | Motor do PSO discreto: <code>_sigmoid</code> , <code>_discrete_velocity_update</code> , <code>_discrete_position_update</code> , <code>run_pso</code> . Inclui <code>PSOConfig</code> dataclass.                                                                         |
| <code>app.py</code>                   | Interface web Streamlit: barra lateral de configuração, execução com barra de progresso, visualizações Plotly (Gantt, radar, curva de convergência, comparação AG vs PSO).                                                                                               |
| <code>src/export.py</code>            | Exportação da rotina para CSV e relatório HTML com gráficos. (Alternativa 1 — Pontuação Extra)                                                                                                                                                                           |
| <code>tests/test_runs.py</code>       | Suite de testes: 5 execuções × 4 perfis × {AG, PSO, Baseline}. Calcula estatísticas, salva <code>test_results.json</code> e gera <code>convergence_chart.png</code> e gráficos por perfil.                                                                               |

### 6.2 Fluxo de Dados

Sim (traços) + GACONFIG/PSOConfig (parâmetros) → `run_ga()/run_pso()` → [`evaluate()` → `activities.py`] × 12.000 vezes → resultado {`best_routine`, `best_fitness`, `best_info`, `history`} → `app.py` (visualização Streamlit/Plotly) → `export.py` (exportação opcional).

### 6.3 Dependências

| Biblioteca              | Versão mínima | Uso                                                              |
|-------------------------|---------------|------------------------------------------------------------------|
| <code>streamlit</code>  | ≥ 1.35.0      | Interface web e componentes interativos                          |
| <code>plotly</code>     | ≥ 5.22.0      | Gráficos interativos (Gantt, radar, curva de convergência)       |
| <code>pandas</code>     | ≥ 2.0.0       | Manipulação de dados tabulares e exportação CSV                  |
| <code>matplotlib</code> | ≥ 3.5.0       | Geração do <code>convergence_chart.png</code> na suite de testes |

### 6.4 Instalação e Execução

16. Clonar o repositório: `git clone https://github.com/andreygarcia/sims-routine-optimizer.git` && `cd sims-routine-optimizer`
17. Instalar dependências: `pip install -r requirements.txt`
18. Iniciar a interface web: `streamlit run app.py` → acessar `http://localhost:8501`
19. Executar a suite de testes: `python tests/test_runs.py` → resultados em `results/`

## 7. Testes, Validação e Resultados

### 7.1 Metodologia

Foram realizadas 5 execuções independentes por algoritmo (AG e PSO) para cada um dos 4 perfis de Sim, totalizando 40 execuções otimizadas e 20 baselines. As sementes utilizadas foram: 42, 7, 123, 999 e 2024. Ambos os algoritmos foram configurados com o mesmo orçamento de avaliações da função objetivo ( $80 \times 150 = 12.000$  avaliações por execução) para garantir comparação justa.

Para cada perfil foram calculados: fitness médio e desvio-padrão, melhoria percentual sobre o baseline, felicidade média normalizada (0–100%), tempo médio de execução em segundos, e curvas de convergência médias (média das 5 execuções por geração/iteração).

### 7.2 Resultados Quantitativos — AG vs PSO vs Baseline

| Perfil      | AG Média $\pm$ DP                     | PSO Média $\pm$ DP                    | Baseline | Felicidade AG / PSO | Tempo AG (s) | Vencedor   |
|-------------|---------------------------------------|---------------------------------------|----------|---------------------|--------------|------------|
| Workaholic  | 1304,14 $\pm$ 25,70                   | <b>1305,28 <math>\pm</math> 9,02</b>  | 0,00     | 63,7% / 63,6%       | 7,45 s       | <b>PSO</b> |
| Social      | <b>1819,04 <math>\pm</math> 45,80</b> | 1802,37 $\pm$ 63,91                   | 0,00     | 62,9% / 63,6%       | 7,52 s       | <b>AG</b>  |
| Criativo    | 1333,76 $\pm$ 18,46                   | <b>1338,19 <math>\pm</math> 10,35</b> | 0,00     | 64,0% / 64,1%       | 7,94 s       | <b>PSO</b> |
| Equilibrado | 1470,23 $\pm$ 21,24                   | <b>1487,77 <math>\pm</math> 14,74</b> | 0,00     | 64,3% / 63,3%       | 8,08 s       | <b>PSO</b> |

*Todos os valores de fitness e tempo são médias de 5 execuções com sementes 42, 7, 123, 999 e 2024. O baseline gerou fitness 0,00 em todos os perfis e execuções — rotinas aleatórias violam sistematicamente as penalidades estruturais (sono, comida, banheiro). Resultados completos em `results/test_results.json`.*

### 7.3 Análise dos Resultados

#### 7.3.1 Superioridade sobre o Baseline

Em todos os 4 perfis e todas as 20 execuções (AG + PSO), ambos os algoritmos superaram completamente o baseline. O baseline obteve fitness 0,00 em 100% das execuções — evidência de que rotinas aleatórias violam sistematicamente as penalidades estruturais mínimas (ausência de sono:  $-100$ , sem comida:  $-75$ , sem banheiro:  $-40$ , sem banho:  $-20 = -235$  de penalidade mínima). AG e PSO atingem entre 1304 e 1819 pontos, comprovando que a otimização evolutiva é imprescindível para este problema.

#### 7.3.2 PSO venceu em 3 de 4 perfis

O PSO foi vitorioso no Workaholic (1305,28 vs 1304,14), Criativo (1338,19 vs 1333,76) e Equilibrado (1487,77 vs 1470,23). **Nos três perfis o PSO também apresentou desvio-padrão menor** — 9,02, 10,35 e 14,74 respectivamente contra 25,70, 18,46 e 21,24 do AG — indicando que o enxame com memória coletiva (gbest) convergiu de forma mais estável nesses perfis.

#### 7.3.3 AG venceu no perfil Social

O AG obteve 1819,04  $\pm$  45,80 contra 1802,37  $\pm$  63,91 do PSO no perfil Social. **O AG apresentou fitness superior, porém com desvio-padrão maior.** A execução com semente 123 do AG atingiu 1847,51 — o maior fitness individual de todo o experimento. O PSO no Social teve o maior desvio-padrão do experimento (63,91), com uma execução (semente 999) caindo para 1717,30, sugerindo sensibilidade à semente nesse perfil.

#### 7.3.4 Custo Computacional

O AG foi consistentemente mais rápido que o PSO em todos os perfis (7,45–8,08 s vs 8,26–8,87 s). O PSO tem overhead adicional na atualização de velocidade por partícula (48 operações de sigmoid por partícula por iteração). Ambos atendem ao requisito de menos de 10 segundos.

### 7.3.5 Convergência: AG gradual, PSO rápido no início

As curvas de convergência em `results/convergence_chart.png` e nos gráficos por perfil mostram padrões distintos: o AG converge gradualmente ao longo das 150 gerações, com melhoria consistente. O PSO converge muito mais rapidamente nas primeiras 20–30 iterações (atingindo 85–90% do valor final), seguido de platôs prolongados com poucas melhorias incrementais. Isso reflete a dinâmica do enxame: exploração inicial intensa via gbest, seguida de exploração fina.

### 7.3.6 Fitness vs. Felicidade Média

As métricas de fitness e felicidade média são complementares mas distintas. O fitness acumula bônus intermediários (moodlets, diversidade) ao longo dos 48 blocos; a felicidade média reflete a média dos estados instantâneos das 8 necessidades ao longo do dia. O componente "felicidade\_média\_diária × 10" na função objetivo garante correlação direta entre as duas métricas — o que é observado na tabela: todos os perfis apresentam felicidade entre 62,9% e 64,3% para ambos os algoritmos.

## 7.4 Evidências Visuais

- `results/convergence_chart.png`: gráfico 2×2 com as curvas de convergência médias (AG vs PSO) para os 4 perfis simultaneamente.
- `results/convergence_workaholic.png`, `convergence_social.png`, `convergence_criativo.png`, `convergence_equilibrado.png`: curvas de convergência isoladas por perfil com maior resolução.
- `results/test_results.json`: arquivo JSON completo com todos os dados brutos — fitnesses individuais por semente, históricos completos de convergência por geração, tempos e felicidades.

## 8. Limitações e Melhorias Futuras

---

### 8.1 Limitações do Modelo

- Taxas de decaimento das necessidades são fixas (NEED\_DECAY), não variando com o estado emocional ou os traços do Sim, como ocorre no jogo original.
- Estado inicial das necessidades fixo em 0.0 nos testes automatizados. Cenários reais com necessidades iniciais distintas podem produzir resultados significativamente diferentes.
- Atividades multi-bloco são atômicas: não é possível interromper uma atividade em andamento (ex: acordar no meio do sono para usar o banheiro).
- O catálogo é estático com 20 atividades. Expandir o catálogo exige edição manual de activities.py.

### 8.2 Limitações Algorítmicas

- O AG pode ficar preso em ótimos locais, visível no desvio-padrão de 45,80 no perfil Social, onde uma execução (semente 999) caiu para 1738,61.
- O PSO discreto tem limitações teóricas em espaços combinatórios de alta dimensionalidade. Para instâncias com mais blocos ou mais atividades, a convergência pode se deteriorar.
- O PSO apresentou variância alta no perfil Social (DP = 63,91) e uma execução de baixa qualidade (1717,30), indicando sensibilidade à semente inicial nesse perfil.
- Não foi realizada análise de sensibilidade formal variando simultaneamente múltiplos parâmetros — os parâmetros foram ajustados empiricamente com base em testes preliminares.

### 8.3 Melhorias Futuras

- Implementar restrições de janelas de horário (ex: trabalho apenas entre 8h–18h, sono apenas entre 22h–8h).
- Permitir configuração do estado inicial das necessidades diretamente na interface Streamlit.
- Implementar algoritmo híbrido AG+PSO: PSO para exploração inicial ampla e AG para refinamento local.
- Adicionar análise de sensibilidade formal variando tamanho de população, gerações, taxa de mutação e taxa de crossover para mapear o impacto na qualidade da solução.
- Implementar persistência de rotinas favoritas em banco de dados local (SQLite) com histórico de execuções.
- Ampliar o modelo de necessidades com decaimento dinâmico dependente de traços e estado emocional.

## 9. Conclusão

---

O Sims Routine Optimizer demonstrou que algoritmos evolutivos — Algoritmo Genético e PSO discreto — resolvem de forma eficiente um problema combinatorial de alta complexidade: a otimização de rotina diária com múltiplas variáveis de estado, decaimento dinâmico, restrições contextuais e função objetivo não-linear. Em todos os 40 experimentos realizados, ambos os algoritmos superaram completamente o baseline aleatório, que obteve fitness 0,00 em todas as execuções por violar sistematicamente as restrições estruturais mínimas.

A comparação técnica avançada (Alternativa 3) entre AG e PSO revelou comportamentos distintos e complementares. O PSO foi superior em 3 dos 4 perfis — Workaholic, Criativo e Equilibrado — com desvio-padrão menor, indicando convergência mais estável pela memória coletiva do enxame. O AG foi superior no perfil Social, atingindo o maior fitness individual do experimento (1847,51 com semente 123), embora com maior variância. Em termos de custo computacional, o AG foi consistentemente mais rápido em todos os perfis (7,45–8,08 s vs 8,26–8,87 s do PSO).

O protótipo atende a todos os requisitos técnicos mínimos da lauda: resolve um problema concreto e não trivial, possui interface funcional e demonstrável via Streamlit, implementa dois algoritmos evolutivos completos com representação, operadores e função de aptidão documentados e justificados, realiza comparação com baseline, executa 5 execuções independentes com sementes distintas, e disponibiliza código, dados e resultados no GitHub. A Alternativa 1 de pontuação extra foi implementada com exportação de resultados em CSV e HTML integrada ao fluxo principal.

Como principal lição, o projeto evidenciou que a qualidade e o design da função de aptidão são os fatores mais críticos para o sucesso de qualquer algoritmo evolutivo: as penalidades estruturais e contextuais, o sistema de moodlets, os bônus de diversidade e equilíbrio, e especialmente o componente de felicidade média diária foram responsáveis por guiar os algoritmos a soluções biologicamente viáveis e contextualmente coerentes — resultados que nenhum operador genético ou partícula isoladamente poderia garantir.

## 10. Declaração de Uso de Inteligência Artificial

| Ferramenta         | Finalidade                                                                                          |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Claude (Anthropic) | Apoio na estruturação do código, revisão da lógica dos algoritmos, geração de documentação e testes |

### Principais Usos

20. Estruturação da arquitetura modular: separação em `src/activities.py`, `src/sim.py`, `src/fitness.py`, `src/genetic_algorithm.py`, `src/psa.py` e `src/export.py`.
21. Implementação dos motores evolutivos: codificação dos operadores do AG (crossover de dois pontos, mutação `replace/swap`, torneio de seleção, elitismo e `repair`) e do PSO discreto (velocidade via `sigmoid`, atualização de posição discreta com `pbest/gbest`).
22. Função de aptidão: formulação das penalidades estruturais e contextuais, sistema de moodlets, bônus de diversidade/equilíbrio e componente de felicidade média diária.
23. Interface Streamlit (`app.py`): construção dos gráficos Plotly (Gantt, radar de necessidades, curva de convergência, comparativo AG vs PSO).
24. Documentação: `README.md`, `slides.md`, `docstrings` e `uso_ia.md` gerados com apoio da IA e revisados pela equipe.
25. Suite de testes (`tests/test_runs.py`): script de comparação estatística AG vs PSO vs Baseline gerado com apoio da IA e validado manualmente.

### Revisão Humana

- Todos os parâmetros do AG e do PSO foram discutidos e ajustados pela equipe com base em execuções reais do protótipo.
- A modelagem de atividades e necessidades (pesos, deltas, penalidades, moodlets, traços de personalidade) foi revisada e calibrada manualmente pelos integrantes.
- Os resultados dos testes comparativos foram analisados criticamente pela equipe, incluindo interpretação das assimetrias entre fitness e felicidade média.
- Nenhum código foi submetido sem leitura, compreensão e aprovação pela equipe.

**A equipe assume total responsabilidade pelo conteúdo deste trabalho. O uso de IA foi instrumental e supervisionado; todas as decisões de projeto, implementação e análise são de autoria da equipe.**



## 11. Referências

---

MITCHELL, M. An Introduction to Genetic Algorithms. Cambridge: MIT Press, 1998.

HOLLAND, J. H. Adaptation in Natural and Artificial Systems. 2. ed. Cambridge: MIT Press, 1992.

KENNEDY, J.; EBERHART, R. Particle swarm optimization. In: Proceedings of ICNN'95 — International Conference on Neural Networks, Perth, 1995. p. 1942–1948.

KENNEDY, J.; EBERHART, R. C. A discrete binary version of the particle swarm algorithm. In: 1997 IEEE International Conference on Systems, Man, and Cybernetics. Orlando, 1997. p. 4104–4108.

SHI, Y.; EBERHART, R. A modified particle swarm optimizer. In: 1998 IEEE World Congress on Computational Intelligence. Anchorage, 1998. p. 69–73.

EIBEN, A. E.; SMITH, J. E. Introduction to Evolutionary Computing. 2. ed. Berlin: Springer, 2015.

The Sims Wiki — Needs mechanics. Disponível em: <https://sims.fandom.com/wiki/Needs>. Acesso em: jun. 2026.